

I/O Streams

I/O Operations

Q74. Write a program to count the number of times a character appears in a file.

Note:

- The character check is **case-insensitive**.
- 'a' and 'A' are considered the same.

Code:

```
import java.io.FileReader;
import java.io.IOException;
import java.util.Scanner;

public class CharacterCount {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the file name: ");
        String fileName = sc.nextLine();

        System.out.print("Enter the character to be counted: ");
        char ch = Character.toLowerCase(sc.next().charAt(0));

        int count = 0;

        try {
            FileReader fr = new FileReader(fileName);

            int data;

            while ((data = fr.read()) != -1) {

                char currentChar = Character.toLowerCase((char) data);

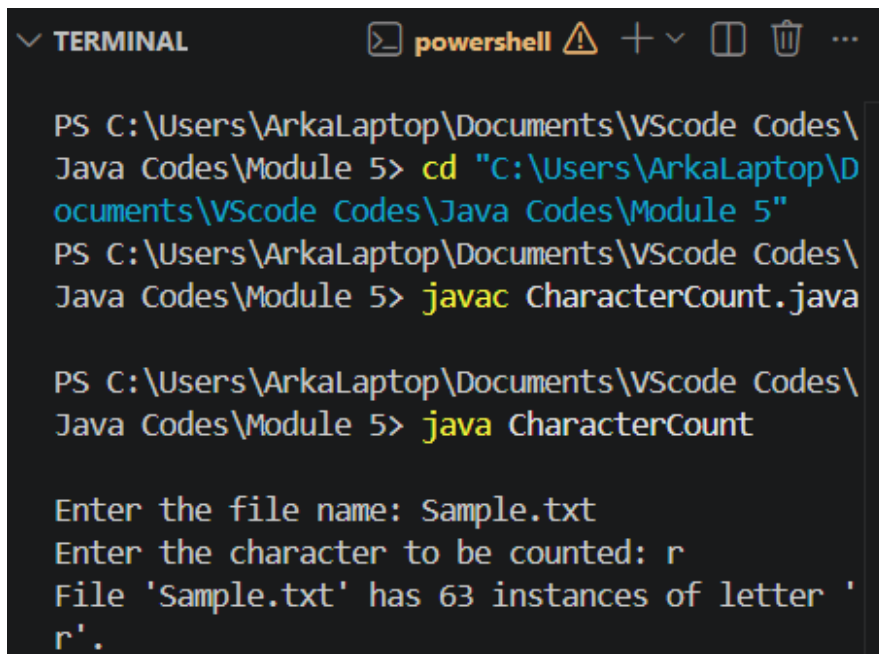
                if (currentChar == ch) {
                    count++;
                }
            }

            fr.close();

            System.out.println("File " + fileName + " has " + count
                               + " instances of letter " + ch + ".");
        }
    }
}
```

```
    } catch (IOException e) {  
        System.out.println("Error: " + e.getMessage());  
    }  
  
    sc.close();  
}  
}
```

Output:



```
✓ TERMINAL powershell ⚠ + - □ 🗑 ...  
  
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 5> cd "C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 5"  
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 5> javac CharacterCount.java  
  
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 5> java CharacterCount  
  
Enter the file name: Sample.txt  
Enter the character to be counted: r  
File 'Sample.txt' has 63 instances of letter 'r'.
```

Q75. Write a program to copy the contents from one file to another and check the output.

Code:

```
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class FileCopy {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter the input file name: ");
        String inputFile = sc.nextLine();

        System.out.print("Enter the output file name: ");
        String outputFile = sc.nextLine();

        try {

            FileReader fr = new FileReader(inputFile);
            FileWriter fw = new FileWriter(outputFile);

            int data;

            while ((data = fr.read()) != -1) {
                fw.write(data);
            }

            fr.close();
            fw.close();

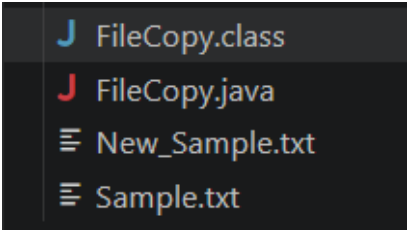
            System.out.println("File is copied.");

        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }

        sc.close();
    }
}
```

Output:

```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 5> javac FileCopy.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 5> java FileCopy
Enter the input file name: Sample.txt
Enter the output file name: New_Sample.txt
File is copied.
```



- FileCopy.class
- FileCopy.java
- New_Sample.txt
- Sample.txt

Q76. Write a program to count the occurrences of each word in an input file and write the word along with its corresponding count into an output file.

Note:

- Words should be **sorted alphabetically** in the output file.
- **Hint:** Use the **Map Collection**.

Code:

```
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Map;
import java.util.Scanner;
import java.util.TreeMap;

public class FileWordCount {

    public static void main(String[] args) {

        if (args.length != 2) {
            System.out.println("Usage: java FileWordCount <inputFile> <outputFile>");
            return;
        }

        String inputFile = args[0];
        String outputFile = args[1];

        TreeMap<String, Integer> wordCount = new TreeMap<>();

        try {

            Scanner fileScanner = new Scanner(new File(inputFile));

            while (fileScanner.hasNext()) {

                String word = fileScanner.next();

                if (wordCount.containsKey(word)) {
                    wordCount.put(word, wordCount.get(word) + 1);
                } else {
                    wordCount.put(word, 1);
                }
            }
        }
```

```
fileScanner.close();

FileWriter writer = new FileWriter(outputFile);

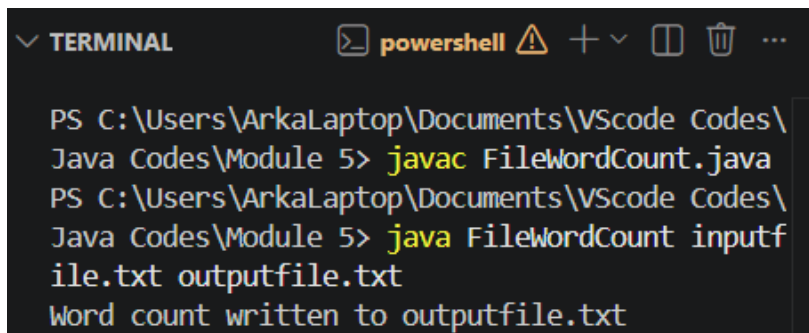
for (Map.Entry<String, Integer> entry : wordCount.entrySet()) {
    writer.write(entry.getKey() + " : " + entry.getValue() + "\n");
}

writer.close();

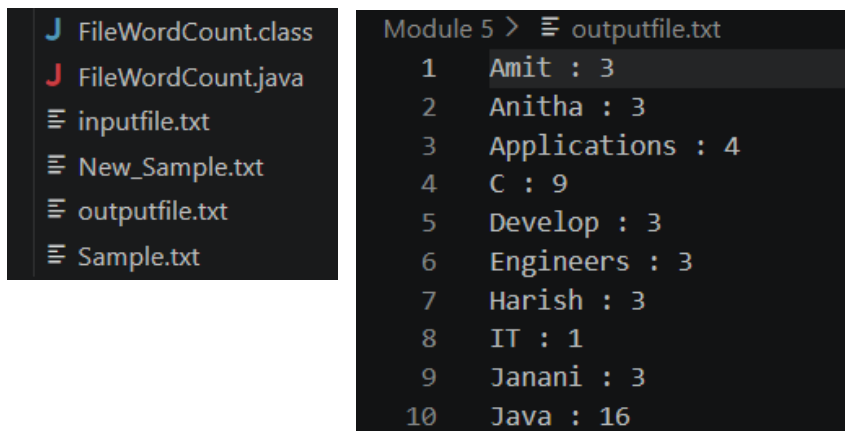
System.out.println("Word count written to " + outputFile);

} catch (IOException e) {
    System.out.println("Error: " + e.getMessage());
}
}
```

Output:



```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 5> javac FileWordCount.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 5> java FileWordCount inputfile.txt outputfile.txt
Word count written to outputfile.txt
```



```
Module 5 > outputfile.txt
1  Amit : 3
2  Anitha : 3
3  Applications : 4
4  C : 9
5  Develop : 3
6  Engineers : 3
7  Harish : 3
8  IT : 1
9  Janani : 3
10 Java : 16
```

Object Serialization

Q77. Create a class called Employee with the following properties:

- name (String)
- dateOfBirth (java.util.Date)
- department (String)
- designation (String)
- salary (double)

Create:

- Getter and Setter methods.
- A no-argument constructor.
- A parameterized constructor.

Then:

1. Create an Employee object.
2. Serialize it into a file named **data**.
3. Deserialize the object from the file.
4. Print all its properties.

Code:

```
import java.io.*;
import java.util.Date;

public class Employee implements Serializable {

    private static final long serialVersionUID = 1L;

    private String name;
    private Date dateOfBirth;
    private String department;
    private String designation;
    private double salary;

    // No-Argument Constructor
    public Employee() {

    }

    // Parameterized Constructor
    public Employee(String name, Date dateOfBirth,
                    String department,
                    String designation,
                    double salary) {

        this.name = name;
        this.dateOfBirth = dateOfBirth;
        this.department = department;
        this.designation = designation;
    }
}
```

```
        this.salary = salary;
    }

    // Getters and Setters

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public Date getDateOfBirth() {
        return dateOfBirth;
    }

    public void setDateOfBirth(Date dateOfBirth) {
        this.dateOfBirth = dateOfBirth;
    }

    public String getDepartment() {
        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }

    public String getDesignation() {
        return designation;
    }

    public void setDesignation(String designation) {
        this.designation = designation;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }

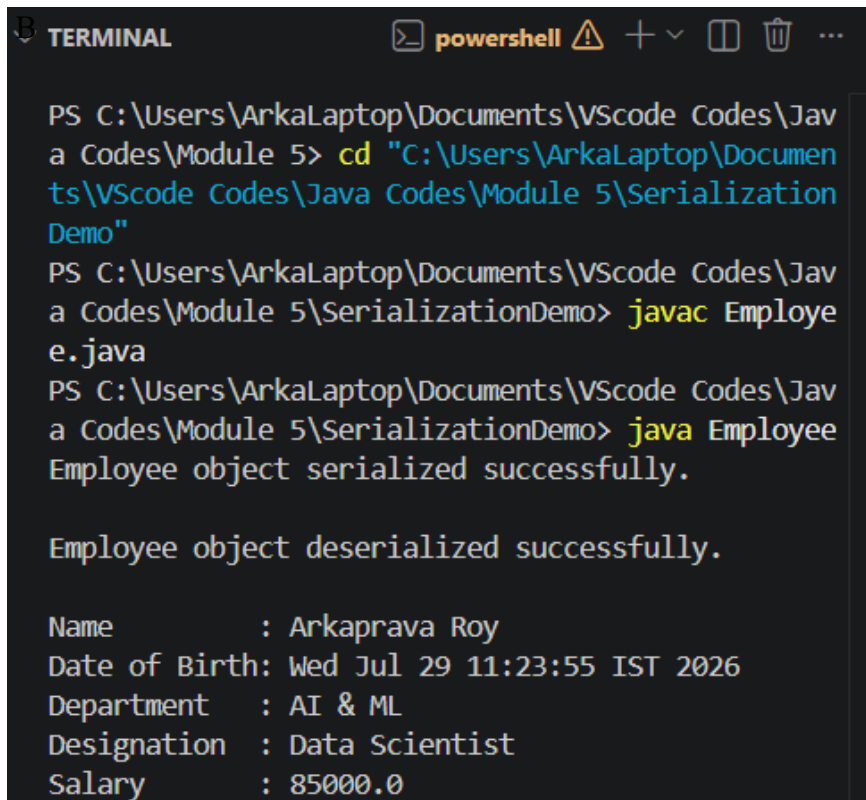
    // Display Method
    public void display() {

        System.out.println("Name      : " + name);
        System.out.println("Date of Birth: " + dateOfBirth);
        System.out.println("Department : " + department);
        System.out.println("Designation : " + designation);
        System.out.println("Salary    : " + salary);
    }

    public static void main(String[] args) {
```

```
try {  
    // Create Employee Object  
    Employee emp = new Employee(  
        "Arkaprava Roy",  
        new Date(),  
        "AI & ML",  
        "Data Scientist",  
        85000);  
  
    // Serialization  
    FileOutputStream fos = new FileOutputStream("data");  
    ObjectOutputStream oos = new ObjectOutputStream(fos);  
  
    oos.writeObject(emp);  
  
    oos.close();  
    fos.close();  
  
    System.out.println("Employee object serialized successfully.\n");  
  
    // Deserialization  
    FileInputStream fis = new FileInputStream("data");  
    ObjectInputStream ois = new ObjectInputStream(fis);  
  
    Employee employee = (Employee) ois.readObject();  
  
    ois.close();  
    fis.close();  
  
    System.out.println("Employee object deserialized successfully.\n");  
  
    employee.display();  
  
} catch (Exception e) {  
    System.out.println(e);  
}  
}
```

Output:



```
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 5> cd "C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 5\Serialization Demo"
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 5\SerializationDemo> javac Employee.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Codes\Module 5\SerializationDemo> java Employee
Employee object serialized successfully.

Employee object deserialized successfully.

Name          : Arkaprava Roy
Date of Birth: Wed Jul 29 11:23:55 IST 2026
Department    : AI & ML
Designation   : Data Scientist
Salary        : 85000.0
```

Module Mini Project

Employee Management System

Create a menu based Java application with the following options.

1. Add an Employee
2. Display All
3. Exit

If option 1 is selected, the application should gather details of the employee like employee name, employee id, designation and salary and store it in a file.

If option 2 is selected, the application should display all the employee details.

If option 3 is selected the application should exit.

Sample Output:

Main Menu

1. Add an Employee
2. Display All
3. Exit

1

Enter Employee ID: 120

Enter Employee Name: Sudhir

Enter Employee Age: 33

Enter Employee Salary: 90000

Main Menu

1. Add an Employee
2. Display All
3. Exit

1

Enter Employee ID :130

Enter Employee Name :Selvan

Enter Employee Age :40

Enter Employee Salary :100000

Main Menu

1. Add an Employee
2. Display All
3. Exit

2

1

2

----Report----

120 Sudhir 33 90000.0

130 Selvan 40 100000.0

----End of Report----

Main Menu

1. Add an Employee
2. Display All
3. Exit

Code:**Employee.java:**

```
import java.io.Serializable;
public class Employee implements Serializable {

    private static final long serialVersionUID = 1L;

    private int empId;
    private String empName;
    private int age;
    private double salary;

    // Default Constructor
    public Employee() {

    }

    // Parameterized Constructor
    public Employee(int empId, String empName, int age, double salary) {
        this.empId = empId;
        this.empName = empName;
        this.age = age;
        this.salary = salary;
    }

    // Getters
    public int getEmpId() {
        return empId;
    }

    public String getEmpName() {
        return empName;
    }

    public int getAge() {
        return age;
    }

    public double getSalary() {
        return salary;
    }

    // Setters
    public void setEmpId(int empId) {
        this.empId = empId;
    }

    public void setEmpName(String empName) {
        this.empName = empName;
    }

    public void setAge(int age) {
        this.age = age;
    }
}
```

```
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }
}
```

EmployeeManagement.java:

```
import java.io.*;
import java.util.ArrayList;
import java.util.Scanner;

public class EmployeeManagement {

    static final String FILE_NAME = "employees.dat";

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        while (true) {

            System.out.println("\nMain Menu");
            System.out.println("1. Add an Employee");
            System.out.println("2. Display All");
            System.out.println("3. Exit");

            System.out.print("Enter your choice: ");
            int choice = sc.nextInt();

            switch (choice) {

                case 1:

                    System.out.print("Enter Employee ID: ");
                    int id = sc.nextInt();
                    sc.nextLine();

                    System.out.print("Enter Employee Name: ");
                    String name = sc.nextLine();

                    System.out.print("Enter Employee Age: ");
                    int age = sc.nextInt();

                    System.out.print("Enter Employee Salary: ");
                    double salary = sc.nextDouble();

                    Employee emp = new Employee(id, name, age, salary);

                    ArrayList<Employee> list = readEmployees();
                    list.add(emp);
                    writeEmployees(list);

                    System.out.println("Employee Added Successfully.");
```

```
        break;

    case 2:

        ArrayList<Employee> employees = readEmployees();

        System.out.println("\n-----Report-----");

        if (employees.isEmpty()) {
            System.out.println("No Employee Records Found.");
        } else {

            for (Employee e : employees) {

                System.out.println(
                    e.getEmpId() + " "
                    + e.getEmpName() + " "
                    + e.getAge() + " "
                    + e.getSalary());

            }

        }

        System.out.println("-----End of Report-----");

        break;

    case 3:

        System.out.println("Exiting the System");
        sc.close();
        System.exit(0);

    default:

        System.out.println("Invalid Choice");
    }

}

}

// Read Employees from File
public static ArrayList<Employee> readEmployees() {

    File file = new File(FILE_NAME);

    if (!file.exists()) {
        return new ArrayList<Employee>();
    }

    try {

        ObjectInputStream ois =
            new ObjectInputStream(
```

```
        new FileInputStream(file));

    ArrayList<Employee> list =
        (ArrayList<Employee>) ois.readObject();

    ois.close();

    return list;
} catch (Exception e) {

    return new ArrayList<Employee>();

}
}

// Write Employees to File
public static void writeEmployees(ArrayList<Employee> list) {

    try {

        ObjectOutputStream oos =
            new ObjectOutputStream(
                new FileOutputStream(FILE_NAME));

        oos.writeObject(list);

        oos.close();

    } catch (Exception e) {

        System.out.println(e);

    }

}
}
```

Output:

```

▼ TERMINAL powershell ⚠ + ▾ 🗑 ...
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Projects> cd "C:\Users\ArkaLaptop\Documents\VScode Codes\Java Projects\EmployeeManagementSystem"

PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Projects\EmployeeManagementSystem> javac Employee.java
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Projects\EmployeeManagementSystem> javac EmployeeManagement.java
Note: EmployeeManagement.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.
PS C:\Users\ArkaLaptop\Documents\VScode Codes\Java Projects\EmployeeManagementSystem> java EmployeeManagement

Main Menu
1. Add an Employee
2. Display All
3. Exit
Enter your choice: 1
Enter Employee ID: 007
Enter Employee Name: Arka
Enter Employee Age: 21
Enter Employee Salary: 90000
Employee Added Successfully.

Main Menu
1. Add an Employee
2. Display All
3. Exit
Enter your choice: 1
Enter Employee ID: 01
Enter Employee Name: Sayan
Enter Employee Age: 22
Enter Employee Salary: 95000
Employee Added Successfully.

Main Menu
1. Add an Employee
2. Display All
3. Exit
Enter your choice: 2

-----Report-----
7 Arka 21 90000.0
1 Sayan 22 95000.0
-----End of Report-----

Main Menu
1. Add an Employee
2. Display All
3. Exit
Enter your choice: 3
Exiting the System
```